

Coding & Solution

Date	15 July 2026
Team ID	SWTID-2026-6128
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/aiswarya08-moravaneni/Emotion-Detection-Learning-Support-Engine
Programming Language(s)	Python 3.11
Framework(s) Used	Streamlit, TensorFlow/Keras, PyTorch/Transformers
Key Features Implemented	• Dual-model (BiLSTM + BERT) emotion classification with confidence scoring and mixed-emotion detection • Gemini-powered personalized guidance generation with an offline mock fallback handler • CSV-based interaction logging • Live Plotly analytics dashboard (emotion distribution + model confidence comparison) • Single-page Streamlit UI tying all modules together
Pending / Incomplete Features	• Real Gemini API call is not yet wired in — MockGeminiHandler currently returns canned responses • The BERT mock (bert_model.py) always returns a fixed prediction rather than genuinely classifying input • Logger schema is inconsistent between the two implementations inside logger.py • requirements.txt does not yet list tensorflow or torch, so a fresh environment setup is incomplete
Setup / Run Instructions	• Create and activate a Python 3.11 virtual environment • Install dependencies (streamlit, plotly, pandas, tensorflow, torch — full list still needs adding to requirements.txt) • Add GEMINI_API_KEY to a .env file • Run streamlit run app.py from the project root

Code Quality Checklist

S.No	Criteria	Status (Yes/No/Partial)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial
4	Error handling is implemented for critical operations	Partial
5	The application runs without critical errors	Partial
6	Code is committed to a version control repository	

Additional Notes / Comments:

Known issues to resolve before this is presented as a fully working, model-driven deliverable: a keyword-based emotion override in app.py currently bypasses the real BiLSTM/BERT prediction for most inputs; the BERT mock always predicts a fixed label rather than varying by input; the label encoder's class list doesn't match the app's five-emotion taxonomy; and the two logging implementations in logger.py write incompatible schemas to the same CSV file.